Studio Nx

P1-02 프론트 공통 JS — 함수 설명 합본 (01~08)

2026-10-06

목차

1. ai-notes/javascript/01_api_refreshAccessToken.md
2. ai-notes/javascript/02_api_apiFetch.md
3. ai-notes/javascript/03_auth_safeNextPath.md
4. ai-notes/javascript/04_auth_loginUrl.md
5. ai-notes/javascript/05_auth_parseLoginHash.md
6. ai-notes/javascript/06_auth_isLoggedIn.md
7. ai-notes/javascript/07_auth_requireLogin.md
8. ai-notes/javascript/08_auth_logout.md

JS 01 — `refreshAccessToken()`

항목	내용
파일	`studionx/frontend/static/js/core/api.js`
작업	P1-02 프론트 공통 JS · PRD 01 FR-105
작성	Claude (AI) · 2026-10-06 · 사용자 검토
다음	02 `apiFetch()`

1. 하는 일

리프레시 쿠키로 Access Token 을 다시 받는다. 성공하면 `true`, 실패하면 `false`.

2. 언제 쓰나

Access Token 은 메모리에만 있어서 **페이지를 옮기면 사라지고**, **30분이 지나면 만료**된다. 그때 로그인을 다시 하지 않고 토큰만 새로 받으려고 쓴다.

상황	부르는 곳
페이지를 열고 첫 API 요청 (토큰이 아직 없음)	`apiFetch()` — 요청 전에 먼저 부름
요청했는데 401 (토큰 만료)	`apiFetch()` — 재발급 후 같은 요청을 한 번 더
로그인이 필요한 페이지에 들어올 때	`isLoggedIn()` → `requireLogin()` (마이페이지 · 내 프로젝트 · 회원 정보 · 시퀀서)
헤더를 로그인 상태로 바꿀 때	`isLoggedIn()` (모든 페이지 헤더)

페이지 코드가 직접 부르는 일은 거의 없다 — 위 함수들을 거친다.

리프레시 쿠키 (`snx_refresh`)

로그인을 오래 유지하려고 서버가 브라우저에 심는 쿠키. 안에 리프레시 토큰이 들어 있다. (서버 쪽은 P1-05 `api/v1/auth.py`)

언제	무슨 일
로그인할 때 (`POST /auth/login`)	서버가 쿠키를 심는다. **자동 로그인 체크** → 3일 유지 (브라우저를 닫아도 남음) · **체크 안 함** → 세션 쿠키 (브라우저를 닫으면 사라짐)
재발급할 때 (`POST /auth/refresh`, 이 함수)	브라우저가 쿠키를 자동으로 보냄 → 서버가 확인하고 새 Access Token 을 줌
로그아웃할 때 (`POST /auth/logout`)	서버가 쿠키를 지운다
그 밖의 API 요청	쿠키가 **실리지 않음** — `path=/api/v1/auth` 로 좁혀 둠

속성	뜻
`HttpOnly`	JS 에서 읽을 수 없음 → 스크립트 공격(XSS)으로 훔칠 수 없음

속성	뜻
`SameSite=Lax`	다른 사이트가 보낸 POST 에는 안 실림 → CSRF 대비
`Secure` (배포)	HTTPS 에서만 전송
`path=/api/v1/auth`	로그인 · 재발급 · 로그아웃 요청에만 실림

토큰	어디에	수명	쓰임
Access Token	메모리 (`api.js` 변수)	30분	모든 API 요청의 `Authorization: Bearer …`
리프레시 토큰	HttpOnly 쿠키	3일 또는 브라우저를 닫을 때까지	Access Token 을 다시 받을 때만

쿠키가 없거나 만료되면 `/auth/refresh` 가 401 → 이 함수가 `false` → 로그인 안 된 상태로 처리.

3. 코드

import (파일 맨 위) `api.js` 는 다른 파일을 import 하지 않는다 (공통 모듈의 맨 아래 단계). 이 함수가 쓰는 `refreshing · setAccessToken · clearAccessToken · API_BASE` 는 같은 파일 안에 있고, `fetch` 는 브라우저 기본 기능이다.

함수

```
export async function refreshAccessToken() {
  if (refreshing) return refreshing;
  refreshing = (async () => {
    try {
      const res = await fetch(`${API_BASE}/auth/refresh`, {
        method: 'POST',
        credentials: 'same-origin',
        headers: { Accept: 'application/json' },
      });
      const data = res.ok ? await res.json().catch(() => null) : null;
      if (!data?.access_token) {
        clearAccessToken();
        return false;
      }
      setAccessToken(data.access_token);
      return true;
    } catch {
      clearAccessToken();
      return false;
    } finally {
      refreshing = null;
    }
  })();
  return refreshing;
}
```

문법 풀이

import 문법	뜻
`export` (함수 앞)	다른 파일이 `import { 함수 }` 로 가져다 쓸 수 있게 내보냄. 없으면 이 파일 안에서만 쓸 수 있음

`if (!data?.access_token)` — "토큰을 못 받았으면"

부분	뜻
`data?.access_token`	**옵셔널 체이닝(`?.`)**: `data` 가 `null`·`undefined` 면 오류 없이 `undefined`, 아니면 `data.access_token`
`!`	값이 없거나(`undefined`·`null`) 빈 문자열이면 `true`

`data`	`data?.access_token`	`!data?.access_token`	결과
`null` (실패 응답 · JSON 아님)	`undefined`	`true`	실패 처리
`{}` (토큰 없음)	`undefined`	`true`	실패 처리
`{ access_token: '' }`	`''`	`true`	실패 처리
`{ access_token: 'eyJ…' }`	`'eyJ…'`	`false`	성공 처리

`?.` 없이 `data.access_token` 이라고 쓰면, `data` 가 `null` 일 때 `TypeError: Cannot read properties of null` 로 멈춘다. 같은 뜻을 길게 쓰면 `if (data === null || data === undefined || !data.access_token)`.

`res.ok ? await res.json().catch(() => null) : null` — 성공 응답(200~299)일 때만 본문을 JSON 으로 읽고, 읽다 실패하면 `null`. 실패 응답이면 읽지 않고 `null`. 그래서 아래 `if` 하나로 모든 실패를 거른다.

4. 흐름

1. 재발급이 이미 진행 중이면 그 Promise 를 돌려준다.
2. `POST /api/v1/auth/refresh` — 쿠키는 브라우저가 자동으로 싣는다.
3. 응답에 `access_token` 이 있으면 저장 → `true`.
4. 401 · 네트워크 오류 · 토큰 없는 응답 → 토큰 지움 → `false`.
5. 끝나면 `refreshing` 을 비운다.

5. 왜 이렇게

선택	이유
진행 중인 Promise 공유 (`refreshing`)	여러 요청이 동시에 401 이어도 재발급은 한 번. 리프레시 토큰을 한 번 쓰고 바꾸는 방식에서도 안전
토큰을 JS 가 넣지 않음	리프레시 토큰은 HttpOnly 쿠키 — JS 에서 안 보임(XSS 대비). `credentials: 'same-origin'` 이면 자동 전송
실패해도 던지지 않음	부르는 쪽(`apiFetch` · `isLoggedIn`)은 성공 여부만 필요
`res.json().catch(() => null)`	JSON 이 아닌 응답도 예외 없이 실패로 처리
`finally` 에서 비움	성공 · 실패와 상관없이 다음 호출은 새로 요청

6. 시험 (fetch 를 가짜로 — `/auth/refresh` 는 P1-05 에서 구현)

경우	결과
200 + `access_token`	`true`, 토큰 저장, 주소 · POST · 쿠키 옵션 확인
401	`false`, 토큰 지움
네트워크 오류	`false`, 토큰 지움, 예외 없음
200 인데 토큰 없음	`false`

경우	결과
동시에 3번 호출	fetch 1번, 셋 다 `true`
끝난 뒤 다시 호출	새로 요청, 새 토큰

JS 02 — `apiFetch(path, options)`

항목	내용
파일	`studionx/frontend/static/js/core/api.js` (도우미 `toApiError` 포함)
작업	P1-02 프론트 공통 JS · PRD 01 FR-105
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	01 `refreshAccessToken()` · 03 `safeNextPath()`

1. 하는 일

서버 API 를 부르는 **하나뿐인** 함수. 토큰을 붙이고, 만료되면 재발급 후 한 번 다시 보내고, 실패는 모두 `ApiError` 로 바꿔 던진다.

2. 언제 쓰나

모든 페이지 · 모듈은 `fetch` 를 직접 쓰지 않고 이 함수(또는 짧은 이름 `api.get` · `api.post` …)를 쓴다.

예	부르는 곳
`api.post('/auth/login', { email, password, remember }, { auth: false })`	로그인 모달 (P1-07)
`api.get('/users/me')`	회원 정보 · 헤더 (P1-10)
`api.post('/users/me/image', formData)`	프로필 이미지 업로드 (P1-10)
`api.get('/projects?page=1')` · `api.post('/projects', { name })`	내 프로젝트 (P1-14)
`api.get('/me/dashboard')`	마이페이지 (P1-17)

옵션	기본	뜻
`method`	`'GET'`	HTTP 방식
`body`	없음	객체 → JSON, `FormData` → 그대로 (파일 업로드)
`auth`	`true`	`false` 면 토큰을 붙이지 않음 — 회원가입 · 로그인 · 문의처럼 로그인 전에 쓰는 API

기대는 원리

원리	내용
`Authorization: Bearer <Access Token>`	서버가 "누가 보냈는지" 아는 방법. 로그인이 필요한 API 는 이 헤더가 없거나 토큰이 만료되면 **401** `UNAUTHORIZED`
서버 오류 형식	모든 실패가 `{ "error": { "code", "message", "details" } }` (P1-03 `core/errors.py`) → 화면은 `err.code` 로 분기, `err.message` 를 보여 줌
`credentials: 'same-origin'`	같은 사이트 요청에 쿠키를 싣는다 — 리프레시 쿠키는 `/api/v1/auth` 아래에만 실림 (01)

3. 코드

import (파일 맨 위) `api.js` 는 다른 파일을 import 하지 않는다 (공통 모듈의 맨 아래 단계). 이 함수가 쓰는 `accessToken · refreshAccessToken · ApiError · toApiError · API_BASE` 는 같은 파일 안에 있고, `fetch` 는 브라우저 기본 기능이다.

함수

```
export async function apiFetch(path, options = {}) {
  const { method = 'GET', body, auth = true } = options;

  const headers = { Accept: 'application/json' };
  let payload;
  if (body instanceof FormData) {
    payload = body;
  } else if (body !== undefined && body !== null) {
    headers['Content-Type'] = 'application/json';
    payload = JSON.stringify(body);
  }

  if (auth && !accessToken) await refreshAccessToken();

  const send = async () => {
    const h = auth && accessToken ? { ...headers, Authorization: `Bearer ${accessToken}` } : headers;
    try {
      return await fetch(API_BASE + path, { method, headers: h, body: payload, credentials: 'same-origi
n' });
    } catch {
      throw new ApiError(0, 'NETWORK_ERROR', 'Network error. Please try again.');
    }
  };

  let res = await send();

  if (res.status === 401 && auth) {
    const err = await toApiError(res);
    if (err.code !== 'UNAUTHORIZED' || !(await refreshAccessToken())) throw err;
    res = await send();
  }

  if (!res.ok) throw await toApiError(res);

  if (res.status === 204) return null;
  const text = await res.text();
  return text ? JSON.parse(text) : null;
}

async function toApiError(res) {
  const data = await res.json().catch(() => null);
  const e = data?.error;
  if (e?.code) return new ApiError(res.status, e.code, e.message ?? '', e.details ?? null);
  return new ApiError(res.status, 'INTERNAL_ERROR', 'Something went wrong. Please try again.');
}
```

문법 풀이

import 문법	뜻
`export` (함수 앞)	다른 파일이 `import { 함수 }` 로 가져다 쓸 수 있게 내보냄. 없으면 이 파일 안에서만 쓸 수 있음

식	뜻
`const { method = 'GET', body, auth = true } = options`	**구조 분해 + 기본값**: `options.method` 가 없으면 `'GET'`, `options.auth` 가 없으면 `true`
`body instanceof FormData`	`body` 가 `FormData` 로 만든 값인지 — 파일 업로드인지 확인
`{ ...headers, Authorization: … }`	**펼침**: `headers` 를 복사하고 한 칸 더한 새 객체. 원래 `headers` 는 그대로라 재시도 때 새 토큰으로 다시 만들 수 있음
`auth && accessToken ? A : B`	둘 다 참이면 A(토큰 붙임), 아니면 B(그대로)
`if (err.code !== 'UNAUTHORIZED' || !(await refreshAccessToken())) throw err`	"만료 오류가 아니거나, 재발급이 실패하면 던진다". `||` 는 앞이 참이면 뒤를 실행하지 않음 → 다른 401(예: 로그인 실패)이면 **재발급을 시도조차 안 함**
`e.message ?? ''`	`??`: 왼쪽이 `null`·`undefined` 일 때만 오른쪽. (`||` 와 달리 `''` · `0` 은 그대로 둠)
`text ? JSON.parse(text) : null`	본문이 비었으면 `null` — `res.json()` 은 빈 본문에서 오류가 나서 글자로 먼저 읽음

4. 흐름

1. 옵션을 읽고 본문을 만든다 (객체 → JSON 문자열, `FormData` → 그대로).
2. 로그인이 필요한 요청인데 토큰이 없으면 먼저 재발급.
3. 보낸다 — 토큰이 있으면 `Authorization` 헤더. 네트워크가 끊기면 `NETWORK_ERROR`(status 0).
4. 401 `UNAUTHORIZED` 면 재발급 → 성공하면 **한 번만** 다시 보낸다. 재발급 실패 · 다른 401 이면 그 오류를 던진다.
5. 그 밖의 실패(4xx · 5xx)는 서버 형식대로 `ApiError`. JSON 이 아니면 `INTERNAL_ERROR`.
6. 성공: 204 · 빈 본문 → `null`, 아니면 JSON.

5. 왜 이렇게

선택	이유
fetch 를 한 곳에서만	토큰 · 재발급 · 오류 형식을 페이지마다 다시 쓰지 않음
`FormData` 면 `Content-Type` 을 안 붙임	브라우저가 `multipart/form-data; boundary=…` 를 직접 붙여야 서버가 파일을 읽음. 직접 쓰면 boundary 가 빠져 깨짐
재시도는 한 번만	재발급 뒤에도 401 이면 무한 반복 대신 바로 실패
401 이라도 `UNAUTHORIZED` 만 재발급	로그인 실패(`INVALID_CREDENTIALS`) 같은 401 은 토큰 문제가 아님
`send()` 안에서 헤더를 매번 만듦	재시도 때 **새** 토큰이 실림
실패를 모두 `ApiError` 로	화면은 `try { … } catch (err) { if (err.code === 'EMAIL_TAKEN') … }` 한 가지 방식으로 처리
실패 본문을 `toApiError` 로 분리	401 처리와 일반 실패가 같은 변환을 씀

6. 시험 (fetch 를 가짜로)

경우	결과
토큰 없음 → `/users/me`	refresh → 요청 순서, `Bearer T1` · `Accept` · `credentials`, 결과 JSON
객체 body (POST)	`Content-Type: application/json`, 본문 `{"name":"A"}`
`FormData` body	그대로, `Content-Type` 안 붙음
`auth: false`	재발급 안 함, `Authorization` 없음
401 `UNAUTHORIZED`	refresh → 같은 요청을 새 토큰으로, 성공 (요청 3번)
401 + 재발급 실패	`ApiError(401, UNAUTHORIZED)`, 토큰 지움
재시도도 401	한 번만 재시도 후 던짐 (요청 3번)
401 `INVALID_CREDENTIALS`	재발급 없이 그대로 던짐 (요청 1번)
422	`code` · `message` · `details.email` 그대로
500 HTML 응답	`INTERNAL_ERROR`
네트워크 오류	status 0 · `NETWORK_ERROR`
204 · 빈 본문 200	`null`

JS 03 — `safeNextPath(next, origin)`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS · PRD 01 FR-105
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	02 `apiFetch()` · 04 `loginUrl()`

1. 하는 일

로그인 후 돌아갈 주소(`next`)가 **우리 사이트 안의 경로**인지 검사한다. 안전하면 그대로, 아니면 `null`.

2. 언제 쓰나

로그인이 필요한 페이지(마이페이지 · 내 프로젝트 · 회원 정보 · 시퀀서)에 로그인 없이 들어오면 `home.html#login?next=/myPage.html` 로 보낸다. 로그인 모달에서 로그인에 성공하면 `next` 로 돌아간다.

상황	부르는 곳
주소의 `#login?next=…` 를 읽을 때	`parseLoginHash()` (05) → `home.js` 로그인 모달
로그인 성공 후 이동하기 직전	로그인 폼 처리 (P1-07) — 한 번 더 확인

막는 공격 — 열린 리다이렉트 (open redirect)

`next` 는 주소창에 그대로 들어 있어 **누구나 바꿀 수 있다**. 검사가 없으면:

1. 공격자가 `https://studionx…/home.html#login?next=https://evil.com/fake-login` 링크를 보낸다.
2. 사용자는 진짜 Studio Nx 주소라 믿고 로그인한다.
3. 로그인 직후 가짜 사이트로 이동 → "비밀번호를 다시 입력하세요" 같은 피싱.

그래서 **같은 사이트 경로만** 허용한다.

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken, getAccessToken, refreshAccessToken } from './api.js';
```

`auth.js` 맨 위의 import 줄이다. **이 함수는 import 한 것을 쓰지 않는다** — `URL` · `location` 은 브라우저 기본 기능이다.

함수

```
export function safeNextPath(next, origin = location.origin) {
  if (typeof next !== 'string' || !next.startsWith('/')) return null;
  if (next.startsWith('//') || next.includes('\\')) return null;
```

```
  try {
    return new URL(next, origin).origin === origin ? next : null;
  } catch {
    return null;
  }
}
```

문법 풀이

import 문법	뜻
`import { a, b } from './api.js'`	`api.js` 가 `export` 한 것 중 `a` · `b` 를 이름 그대로 가져옴. `'./'` 는 같은 폴더. 브라우저는 `<script type="module">` 일 때만 `import` 를 읽는다
`export` (함수 앞)	다른 파일이 `import { 함수 }` 로 가져다 쓸 수 있게 내보냄. 없으면 이 파일 안에서만 쓸 수 있음

식	뜻
`origin = location.origin`	기본값 매개변수 — 보통은 지금 사이트(`http://localhost:8000`). 시험 때만 다른 값을 넣음
`typeof next !== 'string'`	문자열이 아니면(`null` · `undefined` · 숫자) 거부
`'\\'`	자바스크립트 문자열 안의 역슬래시 한 글자 `\`
`new URL(next, origin)`	`next` 를 `origin` 기준으로 해석한 완전한 주소. `new URL('/a', 'http://x')` → `http://x/a`
`.origin === origin`	해석한 주소의 `프로토콜://호스트:포트` 가 우리 사이트와 같은지
`try … catch`	이상한 문자열로 `new URL` 이 오류를 내면 `null`

4. 흐름

1. 문자열이 아니거나 `/` 로 시작하지 않으면 거부 — `https://…`, `javascript:…`, `myPage.html` 이 여기서 걸림.
2. `//` 로 시작하거나 `\` 가 있으면 거부 — 브라우저는 이걸 **다른 사이트 주소**로 읽음.
3. 마지막으로 브라우저와 같은 방식(`new URL`)으로 해석해 우리 사이트인지 확인.

5. 왜 이렇게

선택	이유
`/` 로 시작하는 경로만	가장 단순하고 확실한 기준. 전체 주소(`http://…`)는 우리 사이트여도 거부 — 화면은 경로만 넘김
`//evil.com` 거부	"프로토콜 상대 주소" — 브라우저는 `https://evil.com` 으로 이동
`\` 가 들어가면 거부	브라우저는 `/\evil.com` 의 `\` 를 `/` 로 바꿔 `//evil.com` 으로 읽음
`new URL` 로 한 번 더	위 규칙이 놓친 특이한 형태도 실제 해석 결과로 막음
오류 대신 `null`	부르는 쪽은 `null` 이면 기본 페이지로 보내면 됨

6. 시험

입력	결과
`/myPage.html` · `/studioNx.html?projectId=3` · `/myProject.html#new`	그대로
`//evil.com` · `/\evil.com`	`null`

입력	결과
`https://evil.com/x` · `http://localhost:8000/myPage.html`	`null` (전체 주소는 모두 거부)
`javascript:alert(1)` · `myPage.html` · `''`	`null`
`null` · `undefined` · `42`	`null`
`/%2F%2Fevil.com`	그대로 — 해석하면 `http://localhost:8000/%2F%2Fevil.com`(우리 사이트 안의 경로)이라 안전

JS 04 — `loginUrl(next)`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS · PRD 01 FR-105
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	03 `safeNextPath()` · 05 `parseLoginHash()`

1. 하는 일

로그인 모달로 가는 주소를 만든다. 돌아올 경로가 있으면 `next` 로 붙인다.

`loginUrl('/myPage.html')` → `home.html#login?next=%2FmyPage.html`

2. 언제 쓰나

로그인 화면은 따로 없고 **`home.html` 의 모달**이다 (2026-10-02 결정). 그래서 "로그인하러 가기"는 항상 이 주소로 간다.

상황	부르는 곳
로그인이 필요한 페이지에 로그인 없이 들어옴	`requireLogin()` (07) — `loginUrl(location.pathname + location.search)`
회원가입을 마친 뒤 로그인하러	`join.js` (P1-07) — `loginUrl()`

주소의 `#` 뒤 (해시)

부분	쓰임
`#login`	`home.js` 가 보고 로그인 모달을 자동으로 연다
`?next=…`	로그인 성공 후 돌아갈 경로. `#` 뒤라서 **서버로 보내지지 않고** 브라우저 안에서만 쓰임

보안 — 주소에 `next` 가 보여도 괜찮은가?

한 줄 요약: 주소에 붙는 건 "어디로 가려 했는지"(쪽지)뿐이고, "열쇠"(토큰 · 비밀번호)는 절대 주소에 붙지 않는다.

비유 — 건물 출입

건물	Studio Nx
3층 사무실에 가려는데 출입증이 없다	로그인 없이 `/myPage.html` 에 들어옴
안내 데스크로 보내며 "3층 가려던 분" **쪽지**를 줌	`home.html#login?next=/myPage.html` 로 보냄
안내 데스크에서 신분 확인 → **출입증**을 받음	로그인 → 토큰을 받음
쪽지를 보고 3층으로 안내	`next` 를 읽고 `/myPage.html` 로 이동

쪽지(`next`)는 누가 봐도 괜찮다 — 가려던 층만 적혀 있다. 출입증(토큰)은 주머니(메모리 · HttpOnly 쿠키)에만 넣고 쪽지에는 절대 안 적는다.

순서와 주소창

단계	주소창
① 로그인 없이 마이페이지에 들어옴	`/myPage.html`
② 로그인하러 보냄 — **이때** `next` 가 붙음 (로그인 전)	`home.html#login?next=%2FmyPage.html`
③ 모달에 이메일 · 비밀번호 입력 → 성공	(그대로)
④ `next` 를 검사하고 원래 페이지로	`/myPage.html` — `next` 는 사라짐

걱정과 답

걱정	답
주소에 비밀 정보가 남나?	아니다. 남는 건 경로뿐. 토큰은 메모리 · 쿠키에만
비밀번호가 주소에 붙나?	아니다. 로그인은 `POST` 본문으로 보냄 (주소가 아니라 봉투 안)
서버나 다른 사이트에 새나?	아니다. `#` 뒤는 브라우저 안에만 있고 서버로 가지 않음
누가 `next` 를 가짜 사이트로 바꾸면?	`safeNextPath` 가 우리 사이트 경로만 통과시킴 (03)
`next` 에 스크립트를 넣으면?	`javascript:` 같은 건 거부, 그리고 `next` 는 **이동에만** 쓰고 화면에 HTML 로 넣지 않음
`projectId=3` 이 보이면?	번호는 비밀이 아님. 남의 프로젝트는 서버가 권한으로 막음 (P1-13)

구현 때 지킬 것 (P1-07 로그인 폼)

1. `next` 는 `location.href = next` 처럼 **이동에만** 쓴다 — `innerHTML` 에 넣지 않는다.
2. 로그인 폼은 JS 로 기본 제출을 막고 `apiFetch` 로 보낸다. 폼에 `method="post"` 가 있어 JS 가 실패해도 비밀번호가 주소에 붙지 않는다.

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken, getAccessToken, refreshAccessToken } from './api.js';
```

`auth.js` 맨 위의 import 줄이다. **이 함수는 import 한 것을 쓰지 않는다** — `safeNextPath` · `LOGIN_PAGE` 는 같은 파일 안에 있다.

함수

```
export function loginUrl(next) {
  const path = safeNextPath(next);
  return path ? `${LOGIN_PAGE}#login?next=${encodeURIComponent(path)}` : `${LOGIN_PAGE}#login`;
}
```

문법 풀이

import 문법	뜻
`import { a, b } from './api.js'`	`api.js` 가 `export` 한 것 중 `a` · `b` 를 이름 그대로 가져옴. `'./'` 는 같은 폴더. 브라우저는 `<script type="module">` 일 때만 `import` 를 읽는다

식	뜻
`encodeURIComponent(path)`	주소에서 특별한 뜻이 있는 글자를 `%XX` 로 바꿈. `/` → `%2F`, `?` → `%3F`, `=` → `%3D`, `&` → `%26`
`` `${LOGIN_PAGE}#login?next=${…}` ``	**템플릿 문자열**: `${ }` 안의 값을 끼워 넣은 문자열
`path ? A : B`	안전한 경로가 있으면 A(`next` 붙임), 없으면 B(`#login` 만)

4. 흐름

1. `safeNextPath` 로 경로를 검사한다 (03).
2. 안전하면 인코딩해서 `#login?next=` 뒤에 붙인다.
3. 없거나 안전하지 않으면 `home.html#login` 만.

5. 왜 이렇게

선택	이유
`encodeURIComponent`	경로 안의 `?` · `&` · `=` 가 바깥 `?next=` 의 구분자로 잘못 읽히지 않게. 예: `/myProject.html?q=a&page=2` 의 `&page=2` 가 `next` 밖으로 새지 않음
만들 때도 `safeNextPath`	지금은 우리 코드만 부르지만, 잘못된 값이 들어와도 위험한 주소를 만들지 않음 (읽을 때도 한 번 더 검사)
`LOGIN_PAGE` 상수	로그인 페이지가 바뀌어도 한 곳만 고치면 됨

6. 시험

입력	결과
`/myPage.html`	`home.html#login?next=%2FmyPage.html`
`/studioNx.html?projectId=3`	`…next=%2FstudioNx.html%3FprojectId%3D3`
`/myProject.html?q=a&page=2`	`…next=%2FmyProject.html%3Fq%3Da%26page%3D2` — 되읽으면 원래 경로 그대로
없음 · `''`	`home.html#login`
`https://evil.com` · `//evil.com`	`home.html#login` (`next` 안 붙임)

JS 05 — `parseLoginHash(hash)`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS · PRD 01 FR-105
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	04 `loginUrl()` · 06 `isLoggedIn()`

1. 하는 일

주소의 `#login?next=…` 를 읽는다. **모달을 열지(`open`)** 와 **로그인 후 갈 곳(`next`)** 을 돌려준다.

`parseLoginHash('#login?next=%2FmyPage.html')` → `{ open: true, next: '/myPage.html' }`

2. 언제 쓰나

04 `loginUrl()` 은 로그인 주소를 만들고, 이 함수는 `home.html` 에서 그 주소를 읽는다. 두 함수는 같은 주소 형식(`#login?next=…`)을 쓰므로 함께 바꿔야 한다.

상황	부르는 곳
`home.html` 을 열 때	`home.js` `initLoginModalFromHash()` (P1-07) — `open` 이면 로그인 모달을 열고 `next` 를 기억
로그인 성공 후	로그인 폼 처리 (P1-07) — `next` 가 있으면 그리로, 없으면 기본 페이지로

주소	모달	로그인 후
`home.html`	안 열림	—
`home.html#login` (헤더 Sign In · 가입 완료)	열림	기본 페이지
`home.html#login?next=%2FmyPage.html` (`requireLogin`)	열림	`/myPage.html`

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken, getAccessToken, refreshAccessToken } from './api.js';
```

`auth.js` 맨 위의 import 줄이다. **이 함수는 import 한 것을 쓰지 않는다** — `safeNextPath` 는 같은 파일, `URLSearchParams` · `location` 은 브라우저 기본 기능이다.

함수

```
export function parseLoginHash(hash = location.hash) {
  if (typeof hash !== 'string' || !hash.startsWith('#login')) return { open: false, next: null };
  const rest = hash.slice('#login'.length);
  if (rest && !rest.startsWith('?')) return { open: false, next: null };
  const next = new URLSearchParams(rest.slice(1)).get('next');
```

```
  return { open: true, next: safeNextPath(next) };
}
```

문법 풀이

import 문법	뜻
`import { a, b } from './api.js'`	`api.js` 가 `export` 한 것 중 `a` · `b` 를 이름 그대로 가져옴. `'./'` 는 같은 폴더. 브라우저는 `<script type="module">` 일 때만 `import` 를 읽는다

식	뜻
`hash = location.hash`	기본값 — 지금 주소의 `#` 부터 끝까지 (예: `'#login?next=%2FmyPage.html'`)
`hash.slice('#login'.length)`	앞의 `#login`(6글자)을 잘라낸 나머지 → `''` 또는 `'?next=…'`
`rest && !rest.startsWith('?')`	나머지가 있는데 `?` 로 시작하지 않으면 → `#loginHelp` 같은 **다른** 해시
`rest.slice(1)`	맨 앞 `?` 를 뗀 `next=%2FmyPage.html`
`new URLSearchParams(…).get('next')`	`이름=값&…` 을 읽어 `next` 값을 꺼냄. **인코딩을 자동으로 풀어 줌** — `%2F` → `/`, `%3F` → `?`. 없으면 `null`

hash	rest	결과
`'#login'`	`''`	`{ open: true, next: null }`
`'#login?next=%2FmyPage.html'`	`'?next=%2FmyPage.html'`	`{ open: true, next: '/myPage.html' }`
`'#loginHelp'`	`'Help'`	`{ open: false, next: null }`
`'#features'`	—	`{ open: false, next: null }`

4. 흐름

1. `#login` 으로 시작하지 않으면 → 열지 않음.
2. `#login` 뒤가 비었거나 `?` 로 시작하는지 확인 (`#loginHelp` 거름).
3. `?` 뒤에서 `next` 를 꺼낸다 (인코딩 풀림).
4. `safeNextPath` 로 검사 — 위험하면 `next: null` 이지만 **모달은 연다**.

5. 왜 이렇게

선택	이유
해시(`#`)에 둠	서버로 안 감 · 페이지를 다시 불러오지 않음 (04 보안)
`URLSearchParams`	`&` · `=` · `%XX` 를 직접 자르면 실수하기 쉬움. 브라우저 표준 방식으로 읽음
읽을 때도 `safeNextPath`	주소는 누구나 바꿀 수 있음 — **만들 때 검사했어도 읽을 때 다시**
위험한 `next` 여도 `open: true`	로그인 자체는 정상. 갈 곳만 버리고 기본 페이지로
결과를 객체 하나로	부르는 쪽이 `const { open, next } = parseLoginHash()` 한 줄로 받음

6. 시험

입력	결과
`#login`	열림 · `next` 없음
`#login?next=%2FmyPage.html`	열림 · `/myPage.html`
`#login?next=%2FstudioNx.html%3FprojectId%3D3`	열림 · `/studioNx.html?projectId=3`
`#login?next=%2FmyProject.html%3Fq%3Da%26page%3D2`	열림 · `/myProject.html?q=a&page=2`
`#login?next=https%3A%2F%2Fevil.com` · `…%2F%2Fevil.com`	열림 · `next: null` (풀면 `https://…` · `//evil.com` → 거부)
`#login?foo=1`	열림 · `next: null`
`#loginHelp` · `#features` · `''`	안 열림
인자 없음 (`location.hash` 사용)	`location.hash` 를 읽음
`loginUrl()` 로 만든 주소 → 이 함수로 읽기	원래 경로 그대로 (왕복)

JS 06 — `isLoggedIn()`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS · PRD 01 FR-105
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	05 `parseLoginHash()` · 07 `requireLogin()`

1. 하는 일

지금 로그인 상태인지 `true` · `false` 로 알려 준다. 토큰이 없으면 리프레시 쿠키로 다시 받아 본 뒤 판단한다.

2. 언제 쓰나

상황	부르는 곳
로그인이 필요한 페이지(마이페이지 · 내 프로젝트 · 회원 정보 · 시퀀서)에 들어올 때	07 `requireLogin()` — `false` 면 로그인 화면으로 보낸다
페이지를 열 때, 맨 위 메뉴 줄(헤더) 오른쪽에 어떤 버튼을 보여 줄지 정할 때	`auth-ui.js` `initAuthState()` (P1-07) — 로그인 상태면 **My Page · My Projects** 버튼을, 아니면 **Sign In · Join Us** 버튼을 보여 준다
`home.html` 을 열 때	이미 로그인 상태면 `#login` 모달을 열지 않는다 (P1-07)

헤더 버튼이 바뀌는 예

헤더는 홈 · 가입 · 마이페이지 등 모든 페이지 맨 위에 똑같이 들어가는 메뉴 줄이다 (`static/partials/header.html`). 오른쪽 버튼 네 개가 모두 HTML 에 들어 있고, 이 함수의 결과에 따라 두 개만 보이게 한다.

로그인 상태	보이는 버튼	숨기는 버튼
로그인 안 됨 (`false`)	Sign In · Join Us	My Page · My Projects
로그인 됨 (`true`)	My Page · My Projects	Sign In · Join Us

버튼에는 `data-auth-show="out"` (로그인 안 됐을 때 보임) · `data-auth-show="in"` (로그인했을 때 보임) 표시가 붙어 있다.

로그인 상태를 판단하는 기준

서버에 "로그인 중"이라는 기록은 없다. 브라우저가 가진 토큰으로 판단한다.

브라우저 상태	판단
메모리에 Access Token 이 있다	로그인 상태
Access Token 은 없지만 리프레시 쿠키가 유효하다 (페이지를 옮긴 직후 · 자동 로그인)	재발급에 성공하면 로그인 상태
둘 다 없거나 리프레시 쿠키가 만료되었다	로그인 안 된 상태

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken, getAccessToken, refreshAccessToken } from './api.js';
```

`auth.js` 맨 위의 import 줄이다. 이 함수는 이 중 **`getAccessToken`** · **`refreshAccessToken`** 을 쓴다 (01).

함수

```
export async function isLoggedIn() {
  if (getAccessToken()) return true;
  return refreshAccessToken();
}
```

문법 풀이

import 문법	뜻
`import { a, b } from './api.js'`	`api.js` 가 `export` 한 것 중 `a` · `b` 를 이름 그대로 가져옴. `'./'` 는 같은 폴더. 브라우저는 `<script type="module">` 일 때만 `import` 를 읽는다

식	뜻
`async function`	결과를 Promise 로 돌려주는 함수. 부르는 쪽은 `await isLoggedIn()` 으로 `true` · `false` 를 받는다
`if (getAccessToken())`	토큰 문자열이 있으면 참, `null` 이면 거짓
`return refreshAccessToken();`	재발급 결과(`true` · `false`)를 그대로 돌려준다. `async` 함수 안에서 Promise 를 돌려주면 그 결과가 이 함수의 결과가 된다

4. 흐름

1. 메모리에 Access Token 이 있으면 서버에 묻지 않고 `true`.
2. 없으면 `refreshAccessToken()` 을 부른다 (01).
3. 재발급에 성공하면 `true`, 실패하면 `false`.

5. 왜 이렇게

선택	이유
토큰이 있으면 바로 `true`	같은 페이지 안에서 여러 번 불려도 서버 요청이 생기지 않는다
토큰의 만료 시간은 확인하지 않음	만료된 토큰으로 API 를 부르면 `apiFetch()` 가 401 을 받고 재발급한 뒤 다시 보낸다. 같은 처리를 두 곳에 두지 않는다
재발급은 `refreshAccessToken()` 에 맡김	헤더 버튼을 정하는 코드와 `requireLogin()` 이 페이지를 열 때 동시에 불러도 재발급 요청은 한 번만 나간다 (01 의 진행 중인 Promise 공유)
실패해도 예외를 던지지 않음	네트워크 오류도 "로그인 안 됨"으로 본다. 부르는 쪽은 `true` · `false` 만 확인한다

6. 시험 (fetch 를 가짜로)

경우	결과
메모리에 토큰이 있음	`true`, 서버 요청 0번
토큰 없음 · 리프레시 쿠키 유효	`true`, 재발급 1번, 새 토큰 저장
토큰 없음 · 리프레시 쿠키 없음 (401)	`false`, 토큰 없음
네트워크 오류	`false`, 예외 없음
동시에 두 번 호출 (헤더 버튼 정하기 + `requireLogin`)	둘 다 `true`, 재발급 1번

JS 07 — `requireLogin()`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS · PRD 01 FR-105
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	06 `isLoggedIn()` · 08 `logout()`

1. 하는 일

로그인이 필요한 페이지의 맨 처음에 부른다. 로그인 상태면 `true` 를 돌려주고, 아니면 로그인 화면으로 보낸 뒤 `false` 를 돌려준다.

2. 언제 쓰나

로그인한 사람만 볼 수 있는 페이지 네 곳이 페이지 코드의 맨 처음에 부른다.

페이지	부르는 곳
마이페이지 `myPage.html`	`pages/mypage.js` (P1-17)
내 프로젝트 `myProject.html`	`pages/myproject.js` (P1-14)
회원 정보 `userInfo.html`	`pages/userinfo.js` (P1-10)
시퀀서 `studioNx.html`	`pages/sequencer.js` `init()`

```
const ok = await requireLogin();
if (!ok) return;          // 로그인 화면으로 이동 중 — 아래 코드(대시보드 불러오기 등)를 실행하지 않음
```

실제로 일어나는 일 — 예

로그아웃한 상태에서 주소창에 `localhost:8000/myProject.html#new` 를 직접 입력했을 때:

순서	화면 · 주소창
① 내 프로젝트 페이지가 열리기 시작	`/myProject.html#new`
② `requireLogin()` → 로그인 안 됨	—
③ 홈으로 이동하고 로그인 모달이 열림	`home.html#login?next=%2FmyProject.html%23new`
④ 로그인 성공 (P1-07)	—
⑤ 원래 가려던 곳으로 돌아옴 — 새 프로젝트 모달까지 열림	`/myProject.html#new`

로그인한 상태라면 ② 에서 `true` 가 나오고 페이지가 그대로 이어진다.

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken, getAccessToken, refreshAccessToken } from './api.js';
```

`auth.js` 맨 위의 import 줄이다. **이 함수는 import 한 것을 직접 쓰지 않는다** — `isLoggedIn` · `loginUrl` 은 같은 파일 안에 있고, 그 함수들이 `api.js` 를 쓴다.

함수

```
export async function requireLogin() {
  if (await isLoggedIn()) return true;
  location.replace(loginUrl(location.pathname + location.search + location.hash));
  return false;
}
```

문법 풀이

import 문법	뜻
`import { a, b } from './api.js'`	`api.js` 가 `export` 한 것 중 `a` · `b` 를 이름 그대로 가져옴. `'./'` 는 같은 폴더. 브라우저는 `<script type="module">` 일 때만 `import` 를 읽는다

식	뜻
`await isLoggedIn()`	로그인 상태 확인이 끝날 때까지 기다린 뒤 `true` · `false` 를 받음 (06)
`location.pathname`	주소의 경로 — `/myProject.html`
`location.search`	`?` 부터 — `?projectId=3` (없으면 `''`)
`location.hash`	`#` 부터 — `#new` (없으면 `''`)
`location.replace(url)`	`url` 로 이동하되 **지금 페이지를 방문 기록에서 지움**

4. 흐름

1. `isLoggedIn()` 이 `true` 면 바로 `true` — 페이지가 계속 진행된다.
2. 아니면 지금 주소(경로 + 쿼리 + 해시)로 로그인 주소를 만든다 (04 `loginUrl()`).
3. 그 주소로 `location.replace` 하고 `false` 를 돌려준다.

5. 왜 이렇게

선택	이유
`location.href` 대신 `location.replace`	`href` 로 이동하면 로그인 화면에서 뒤로 가기를 눌렀을 때 마이페이지로 돌아갔다가 **다시 로그인 화면으로 튕긴다** (반복). `replace` 는 마이페이지를 기록에서 지워 뒤로 가기가 그 전 페이지로 간다
쿼리 · 해시까지 넣음	`?projectId=3`(어떤 프로젝트) · `#new`(새 프로젝트 모달)가 빠지면 로그인 후 엉뚱한 화면이 된다. 원래 템플릿은 경로 + 쿼리였는데 해시도 더함
`false` 를 돌려줌	이동은 바로 끝나지 않는다. 그 사이 페이지 코드가 API 를 부르지 않도록 `if (!ok) return;` 으로 멈추게 한다
판단은 `isLoggedIn()` 에 맡김	토큰 · 재발급 처리를 다시 쓰지 않는다

6. 시험 (fetch · location 을 가짜로)

경우	결과
토큰 있음	`true`, 이동 없음
토큰 없음 · 리프레시 쿠키 유효	`true`, 이동 없음
로그인 안 됨 · `/myPage.html`	`false`, `home.html#login?next=%2FmyPage.html` 로 이동
`/studioNx.html?projectId=3`	`…next=%2FstudioNx.html%3FprojectId%3D3`
`/myProject.html#new`	`…next=%2FmyProject.html%23new`
위 주소를 `parseLoginHash()` 로 다시 읽기	`/myProject.html#new` 그대로

JS 08 — `logout()`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS · PRD 01 FR-104
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전	07 `requireLogin()` — P1-02 의 마지막 함수

1. 하는 일

로그아웃한다. 서버에 리프레시 쿠키를 지워 달라고 요청하고, 메모리의 Access Token 을 지운 뒤 홈으로 이동한다.

2. 언제 쓰나

마이페이지 · 내 프로젝트 · 회원 정보 화면의 **왼쪽 메뉴 맨 아래 `Log out` 버튼**(`#logout-button`)을 누를 때 부른다.

페이지	부르는 곳
마이페이지 `myPage.html`	`pages/mypage.js` (P1-17)
내 프로젝트 `myProject.html`	`pages/myproject.js` (P1-14)
회원 정보 `userInfo.html`	`pages/userinfo.js` (P1-10)

누르면 일어나는 일 — 예

순서	화면 · 브라우저
① 마이페이지에서 `Log out` 누름	—
② 서버가 리프레시 쿠키(`snx_refresh`)를 지움	다음에 페이지를 열어도 재발급이 안 됨
③ 메모리의 Access Token 을 지움	지금 페이지에서도 API 를 부를 수 없음
④ 홈으로 이동	`home.html` — 헤더 오른쪽이 **Sign In · Join Us** 버튼으로 바뀜
⑤ 뒤로 가기를 누름	마이페이지가 아니라 그 전에 보던 페이지로 감

토큰이 두 개라서 로그아웃도 두 곳에서 지운다.

토큰	지우는 곳
리프레시 토큰 (HttpOnly 쿠키)	서버 — JS 는 이 쿠키를 읽거나 지울 수 없음
Access Token (메모리)	이 함수 — `clearAccessToken()`

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken, getAccessToken, refreshAccessToken } from './api.js';
```

`auth.js` 맨 위의 import 줄이다. 이 함수는 이 중 **`api`** · **`clearAccessToken`** 을 쓴다. `LOGIN_PAGE` 는 같은 파일 안에 있다.

함수

```
export async function logout() {
  try {
    await api.post('/auth/logout', undefined, { auth: false });
  } catch {
    // 이미 만료 · 네트워크 오류여도 화면에서는 로그아웃을 계속함
  }
  clearAccessToken();
  location.replace(LOGIN_PAGE);
}
```

문법 풀이

import 문법	뜻
`import { a, b } from './api.js'`	`api.js` 가 `export` 한 것 중 `a` · `b` 를 이름 그대로 가져옴. `'./'` 는 같은 폴더. 브라우저는 `<script type="module">` 일 때만 `import` 를 읽는다

식	뜻
`api.post('/auth/logout', undefined, { auth: false })`	본문 없이(`undefined`) POST. `auth: false` 라서 토큰을 붙이지 않고, 토큰이 없어도 재발급을 먼저 시도하지 않음 (02)
`try { … } catch { }`	요청이 실패해도 오류로 멈추지 않고 다음 줄로 넘어감. `catch` 뒤의 `(err)` 는 쓰지 않으면 생략할 수 있음
`location.replace(LOGIN_PAGE)`	`home.html` 로 이동하면서 지금 페이지를 방문 기록에서 지움

4. 흐름

1. `POST /api/v1/auth/logout` — 서버가 리프레시 쿠키를 지운다.
2. 실패해도 그대로 진행한다.
3. 메모리의 Access Token 을 지운다.
4. `home.html` 로 이동한다 (방문 기록에서 지금 페이지를 지움).

5. 왜 이렇게

선택	이유
`auth: false`	서버는 토큰 확인 없이 쿠키만 지운다 (P1-05 템플릿). 토큰이 없을 때 재발급을 먼저 하면 **로그아웃하려다 오히려 새 토큰을 받는** 일이 생긴다
요청이 실패해도 계속	사용자는 로그아웃 버튼을 눌렀다. 네트워크가 끊겨도 화면에서는 로그아웃된 상태가 되어야 한다
`location.href` 대신 `location.replace`	`href` 면 뒤로 가기로 마이페이지가 다시 보일 수 있다(브라우저가 저장해 둔 화면). `replace` 는 그 기록을 지운다. 템플릿은 `href` 였는데 바꿈
메모리 토큰은 마지막에 지움	요청을 보내는 동안에는 필요 없지만, 실패 여부와 상관없이 반드시 지우도록 `try` 바깥에 둠

6. 시험 (fetch · location 을 가짜로)

경우	결과
정상 (204)	`POST /api/v1/auth/logout` 1번 · `credentials` · `Authorization` 없음 · 토큰 지움 · `home.html` 로 replace
토큰이 이미 없음	재발급 시도 없이 로그아웃 요청 1번
서버 오류 500	예외 없음, 토큰 지움, 이동
네트워크 오류	예외 없음, 토큰 지움, 이동